Alexander James Slater

alexslater004@gmail.com | +44 7749 901971 | linkedin.com/in/alexslater1 | github.com/alexslater1

Education

Imperial College London Sept 2022 – June 2026
Computing MEng: First-Class Honours

AWS Cloud Bootcamp August 2026
EC2/ECS, Auto scaling, S3/EBS/EFS storage, Load balancing and Multi-AZ hosting

Monk's Walk School Sept 2020 – June 2022
*A-Levels: A*A*A*A* (Maths, Further Maths, Comp Sci, Physics)*

Experience

Solidatus March 2025 – September 2025
Software Engineer Intern *C# .NET, TypeScript, React, MongoDB, JanusGraph (Gremlin)*

- Built a trace query language that lets users trace data both upstream and downstream, translating user queries into graph engine requests
- Cut datamap load times by 90% by profiling the model subset calculator, folding the query loop into a single traversal, replacing full-graph loads with indexed filtering on entity roots and caching expanded subsets
- Created datamap display rules for the data lineage platform, matching entities against conditions in C# and applying filtering, highlighting and tagging in React

Projects

SolverLNSimple: Layered Queuing Network Solver | *Java, MEng Individual Project* 2026

- Built a layered queuing network solver for predicting distributed-system performance, translating mathematical models into code
- Profiled and rewrote the Mean Value Analysis kernel with primitive arrays in place of sparse-matrix operations, cutting large-model solve time from 60s to 0.1s per iteration
- Achieved 131x faster solves and 9x fewer iterations than LINE's SolverLN across 78 models, within 5% accuracy; an ablation study showed that its convergence control accounted for 73.6% of the iteration gap

WACC Compiler | *Scala* 2024

- Implemented WACC parsing and semantic checks in Scala, including type checking and scoped symbol resolution
- Generated x86-64 assembly for functions, arrays, pairs, and control flow, with stack and register allocation and runtime checks
- Optimised compiled programs with AST-level constant propagation, folding and control-flow analysis, removing dead branches, uncalled functions and unreachable code to cut generated assembly by 8%

PintOS Operating System Kernel | *C* 2024

- Built lazy page loading and a frame table for eviction and swapping, with stack growth and memory-mapped files
- Added argument passing and process creation/wait handling for user programs, and implemented system calls for process control and I/O
- Implemented priority-based kernel thread scheduling with priority donation and a multilevel feedback queue

Soundcheck: Media Sharing App | *Java, Spring Boot, PostgreSQL, React Native, AWS* 2025

- Built a Spring Boot REST API on PostgreSQL and React Native client for sharing concert clips and finding gigs
- Designed a scalable AWS video pipeline: presigned multi-part uploads straight to S3, with SQS feeding FFmpeg workers transcoding clips to HLS on Fargate, deployed separately and scaled on queue depth
- Cut feed p95 latency from 16s to 12ms on a 1M-post load test by rewriting an OR filter as indexed UNIONs
- Trained a 12-feature logistic feed ranker in Python, raising simulated NDCG@10 from 0.574 to 0.647

Technical Skills

Kotlin, Java, Python, TypeScript/JavaScript, C# (.NET), Scala, C, C++, PostgreSQL, Spring Boot, React, Next.js, AWS